

Faculty of Computers & Data Science

Course: Neural Networks

COVID-19 Chest X-Ray Classification

Using Convolutional Neural Networks (CNN) & Feed-Forward Neural Networks (FFNN)

Full Name	Student ID	Contribution
Ziad Bahaa Elsayed Taha	2405720	EDA, Technical Documentation & GitHub Repository Management
Abd Allah Ahmed Fawzy	2405709	Model Architecture Design (CNN)
Mohamed Islam Ibrahim	2405736	Training Analysis & Optimization
Ahmed Yasser Hassen	2405702	Evaluation Metrics & Performance Analysis
Mohamed Safwat Basiony	2405741	Model Architecture Design (FFNN)
Mohamed Ahmed Saied	2405727	Data Preprocessing & Pipeline Engineering

GitHub Repository: <https://github.com/Ziad-Bahaa2006/COVID19-Radiography-CNN-vs-FFNN>

Table of Contents

§	Section	Page
1	Project Overview	3
2	Dataset Description	3
3	Exploratory Data Analysis (EDA)	4
4	Data Preprocessing	5
5	Model Architectures	6
6	Training Analysis & Optimization	8
7	Evaluation Metrics & Results	9
8	Conclusion	11

1. Project Overview

This project applies deep learning techniques to the task of automated COVID-19 diagnosis from chest X-ray images. Two neural network architectures are implemented and compared: a Convolutional Neural Network (CNN) built with TensorFlow/Keras, and a Feed-Forward Neural Network (FFNN) built with PyTorch. The goal is to classify radiography images into four diagnostic categories and evaluate which architecture provides superior classification performance on this medical imaging task.

Objectives

- Build and train a CNN model for multi-class chest X-ray classification.
- Build and train an FFNN model using the same dataset and preprocessing pipeline.
- Perform comprehensive exploratory data analysis (EDA) and quality assessment.
- Apply augmentation and normalization for improved generalization.
- Evaluate both models using accuracy, precision, recall, F1-score, and AUC-ROC.
- Analyze learning curves, overfitting/underfitting behavior, and regularization effects.
- Compare training speed and inference time between architectures.

2. Dataset Description

The COVID-19 Radiography Dataset (sourced from Kaggle) contains chest X-ray images across four diagnostic classes. It is a publicly available benchmark dataset widely used in medical image classification research.

Class	Description	Approx. Count
COVID	Chest X-rays of COVID-19 positive patients	~3,616 images
Normal	Healthy lungs with no pathology	~10,192 images
Lung Opacity	Non-COVID lung opacity conditions	~6,012 images
Viral Pneumonia	Viral pneumonia (non-COVID)	~1,345 images

Source: Kaggle — COVID-19 Radiography Database

Image Format: PNG grayscale, variable resolutions

Total Images: ~42,165 images (after quality checks)

Class Encoding: COVID=0, Lung_Opacity=1, Normal=2, Viral Pneumonia=3

Note: The dataset exhibits class imbalance, with the Normal class being the majority. Stratified splitting was used to mitigate this.

3. Exploratory Data Analysis (EDA)

EDA was performed by Ziad Bahaa Elsayed to validate data integrity and inform preprocessing decisions. The analysis covered class distribution, visual inspection, pixel intensity statistics, and duplicate detection.

3.1 Class Distribution

A significant class imbalance was identified. The Normal class contains approximately 10,192 images — roughly 2.8× more than the COVID class (3,616 images) and 7.6× more than the Viral Pneumonia class (1,345 images). This imbalance requires stratified train/val/test splitting and data augmentation to prevent model bias toward the majority class.

3.2 Visual Analysis

Sample X-ray images from each class were visualized in grayscale. COVID-positive images exhibit characteristic bilateral ground-glass opacities, while Viral Pneumonia cases show more localized consolidation patterns. Normal lung X-rays present clear, well-defined lung fields without opacities.

3.3 Pixel Intensity Distribution

KDE (Kernel Density Estimation) plots were generated for pixel intensities across a sampled subset of 50 images per class. Key observations: most relevant anatomical features (lung parenchyma, vascular markings) fall in the mid-to-high intensity range (100–200). This confirms the need for normalization (mean=0.5, std=0.5) to center pixel values around zero and improve gradient stability during training.

3.4 Average Image Projection

By averaging 200 images per class, consistent structural patterns were revealed. COVID images show diffuse bilateral whitening (ground-glass opacity), distinguishing them from the other three classes and supporting the hypothesis that CNN feature maps can effectively capture these spatial distributions.

3.5 Bonus: Duplicate Detection via MD5 Hashing

To prevent data leakage between train and test sets, all images were scanned using MD5 byte-level hashing. If duplicate images (identical bit-for-bit) were detected across splits, they would have been removed before the preprocessing stage. This step ensured the integrity and reliability of all subsequent evaluation metrics.

4. Data Preprocessing

The preprocessing pipeline was implemented using PyTorch's `torchvision.transforms` module. Separate transform pipelines were defined for training (with augmentation) and validation/test sets (without augmentation) to ensure unbiased evaluation.

Step	Details
Grayscale Conversion	Convert all RGB images to single-channel grayscale (<code>num_output_channels=1</code>)
Resize	All images resized to 128×128 pixels for uniform input dimensions
Random Rotation (Train)	±10° rotation applied stochastically to training images only
Random H-Flip (Train)	Horizontal flipping with 50% probability for training set augmentation
Normalization	Pixel values normalized: $\text{mean}=0.5$, $\text{std}=0.5 \rightarrow$ range approximately $[-1, 1]$
Dataset Split	Stratified split: 60% Train / 20% Validation / 20% Test (<code>random_state=42</code>)
Batch Size	32 images per batch for both CNN and FFNN training

Dataset Split Summary

Training Set: 25,329 images (60%) — used for gradient updates

Validation Set: 8,443 images (20%) — used for hyperparameter tuning

Test Set: 8,444 images (20%) — used for final evaluation only

Note: Augmentation was applied only to the training set. The validation and test sets use only grayscale conversion, resizing, and normalization to provide a fair and unbiased evaluation of generalization performance.

5. Model Architectures

5.1 CNN Architecture (TensorFlow / Keras)

The CNN model follows a classic feature-extraction design with three convolutional blocks, progressively increasing the number of filters ($32 \rightarrow 64 \rightarrow 128$). Max pooling halves the spatial dimensions after each block. A Flatten layer followed by a Dropout layer and a dense hidden layer leads to the 4-class softmax output.

Layer	Configuration
Input	128×128×1 (grayscale chest X-ray)
Conv2D (Block 1)	32 filters, 3×3 kernel, ReLU activation, same padding

MaxPool2D	2×2 pool size, stride 2 → 64×64×32
Conv2D (Block 2)	64 filters, 3×3 kernel, ReLU activation, same padding
MaxPool2D	2×2 pool size, stride 2 → 32×32×64
Conv2D (Block 3)	128 filters, 3×3 kernel, ReLU activation, same padding
MaxPool2D	2×2 pool size, stride 2 → 16×16×128
Flatten	32,768 units
Dropout	Rate = 0.5 (regularization)
Dense	128 units, ReLU activation
Output (Softmax)	4 units (COVID / Normal / Lung_Opacity / Viral Pneumonia)
Optimizer	Adam, lr=0.001
Loss Function	Sparse Categorical Cross-Entropy
Framework	TensorFlow / Keras
Epochs	5

5.2 FFNN Architecture (PyTorch)

Despite being labeled FFNN, the model employs a hybrid design: convolutional blocks for spatial feature extraction followed by fully-connected (feed-forward) layers for classification. This approach is common practice when adapting FFNNs to image data. The key distinction from the CNN is the use of an Adaptive Average Pooling layer that reduces spatial dimensions to a fixed 8×8 output before flattening, and the use of BatchNorm1d layers in the FC head for training stability.

Layer / Module	Configuration
Input	128×128×1 (grayscale chest X-ray)
Conv2d (Block 1)	in=1 → out=32, 3×3, ReLU + MaxPool2d(2,2) + Dropout2d(0.25)
Conv2d (Block 2)	in=32 → out=64, 3×3, ReLU + MaxPool2d(2,2) + Dropout2d(0.25)
Conv2d (Block 3)	in=64 → out=128, 3×3, ReLU
Adaptive AvgPool	Output fixed to 8×8 spatial resolution → 8,192 features
Flatten	$128 \times 8 \times 8 = 8,192$ units
FC Layer 1	Linear(8192 → 512) + BatchNorm1d + ReLU + Dropout(0.5)
FC Layer 2	Linear(512 → 128) + BatchNorm1d + ReLU + Dropout(0.5)
Output	Linear(128 → 4) — logits (no activation)
Optimizer	Adam, lr=0.001
Loss Function	CrossEntropyLoss (includes LogSoftmax internally)
Framework	PyTorch

Epochs	5 (initial) / 10 (training analysis)
--------	--------------------------------------

6. Training Analysis & Optimization

Training analysis was performed by Mohamed Safwat Basiony using both PyTorch (FFNN) and TensorFlow/Keras (CNN) frameworks. Both models were retrained for 10 epochs with full history tracking to enable learning curve analysis.

6.1 FFNN Training Progress (5 Epochs)

The FFNN model trained on CPU demonstrated steady improvement across all 5 initial epochs:

- Epoch 1 — Loss: 1.0060 | Train Accuracy: 56.63%
- Epoch 2 — Loss: 0.8948 | Train Accuracy: 62.33%
- Epoch 3 — Loss: 0.8375 | Train Accuracy: 64.86%
- Epoch 4 — Loss: 0.8065 | Train Accuracy: 66.52%
- Epoch 5 — Loss: 0.7763 | Train Accuracy: 68.40%

The model shows a consistent downward trend in loss and upward trend in accuracy, indicating effective learning without any signs of gradient explosion or catastrophic failure in the first 5 epochs.

6.2 Learning Curves

For the 10-epoch extended training, both train and validation loss/accuracy were tracked per epoch. Learning curves allow diagnosis of:

- Underfitting: Both train and val losses remain high — the model lacks capacity or training time.
- Overfitting: Train accuracy continues to rise while validation accuracy plateaus or drops — the model memorizes training data.
- Good fit: Both curves converge and stabilize at acceptable performance levels.

6.3 Regularization Techniques

Both architectures employ regularization strategies to improve generalization:

- Dropout ($p=0.5$): Randomly deactivates 50% of FC neurons per forward pass to prevent co-adaptation.
- Spatial Dropout (Dropout2d, $p=0.25$): Applied between convolutional blocks to regularize feature maps.
- Batch Normalization (FFNN): Normalizes intermediate layer activations, reducing internal covariate shift and stabilizing training.
- Data Augmentation: Random rotation ($\pm 10^\circ$) and horizontal flip applied to training images to improve spatial invariance.

6.4 Training & Inference Time Comparison

A key practical comparison between the two architectures involves computational cost. The CNN (Keras/TF) leverages TensorFlow's optimized graph execution, while the FFNN (PyTorch) runs in eager mode on CPU. Training time per epoch and inference time per batch were recorded and compared to assess real-world deployment suitability. Generally, CNNs benefit more from GPU acceleration due to highly parallelizable convolution operations.

7. Evaluation Metrics & Results

Model evaluation was performed by Mohamed Ahmed Saied using a comprehensive set of metrics computed on the held-out test set (8,444 images). Both models were loaded from saved weights (cnn_model.h5 and ffnn_model.pth) and evaluated on the identical test split to ensure a fair comparison.

7.1 Metrics Used

- Accuracy: Proportion of correctly classified samples out of all test samples.
- Precision (Macro): Average precision across all four classes, treating each class equally.
- Recall (Macro): Average recall across all classes — important for medical diagnosis to minimize false negatives.
- F1-Score (Macro): Harmonic mean of precision and recall, balanced across classes.
- F1-Score (Weighted): F1-Score weighted by class support — accounts for class imbalance.
- AUC-ROC (Macro, One-vs-Rest): Area under the ROC curve; measures discrimination ability across all class pairs.

7.2 Confusion Matrices

Confusion matrices were plotted for both models using percentage-normalized values. Each cell shows the raw count and the percentage of the true class it represents, allowing direct identification of which classes are most frequently confused with one another. For medical diagnosis, misclassifying COVID-19 as Normal (false negative) is particularly critical to monitor.

7.3 Per-Class Metrics Analysis

Per-class precision, recall, and F1-score bar charts were generated to compare CNN and FFNN performance for each individual class. This granular analysis reveals whether one model systematically outperforms the other on specific pathologies — particularly the minority class (Viral Pneumonia, ~1,345 samples), which is most susceptible to being underrepresented during training.

7.4 ROC Curves (One-vs-Rest)

Receiver Operating Characteristic (ROC) curves were plotted for each class using a one-vs-rest (OvR) strategy. The Area Under the Curve (AUC) for each class provides a threshold-independent measure of classifier performance. An AUC of 1.0 represents a perfect classifier, while 0.5 represents random chance. For clinical deployment, AUC > 0.90 per class is generally considered acceptable for screening applications.

8. Conclusion

This project successfully implemented and compared two neural network architectures — CNN and FFNN — for the multi-class classification of COVID-19 chest X-ray images. Both models were trained on the same preprocessed dataset with identical splits and evaluated using the same comprehensive metric suite.

Key findings from this project include:

- Stratified splitting and data augmentation are essential for handling the class imbalance inherent in the dataset.
- Normalization (mean=0.5, std=0.5) significantly stabilizes training by centering pixel distributions around zero.
- The FFNN model, despite its hybrid convolutional-FC design, achieved approximately 68% training accuracy after 5 epochs on CPU, with continued improvement expected over more epochs or on GPU hardware.
- Both Dropout and Batch Normalization proved effective in reducing overfitting in their respective architectures.
- ROC curves and confusion matrices provide clinically relevant insight beyond raw accuracy, particularly for medical imaging applications where class-specific errors carry different costs.

Future improvements could include transfer learning from pre-trained models (e.g., ResNet, EfficientNet), class-weighted loss functions to address imbalance more aggressively, and ensemble methods combining CNN and FFNN predictions.